

M12 — Maquinas de estado del CubeSat INTISAT

Mision, subsistemas y matriz operacional
Documentacion tecnica profunda con diagramas verificados

Equipo INTISAT

Mayo 2026

Prefacio

Este manual es la version impresa, profunda y autocontenida del archivo HTML `state_machines_cubesat.ht`. Es el documento que vas a llevar a la reunion donde se definen los modos de operacion del INTISAT con los lideres de cada subsistema.

Por que existe este manual. Un CubeSat NO se opera con “codigo plano”. Se opera con *maquinas de estado*: un conjunto de modos discretos, con transiciones bien definidas, donde *en cualquier instante* el operador de tierra sabe exactamente en que estado esta el satellite y que comandos puede aceptar. Esto es diferente al desarrollo de software de aplicacion en Tierra y es una de las cosas que mas confunde a equipos que vienen de DevOps o de sistemas empresariales.

Que es nuevo respecto al HTML. Tres cosas:

1. **Diagramas regenerados en matplotlib** con posicionamiento explicito (no Mermaid auto-layout). Cada caja esta donde la pusimos, las flechas curvas tienen radios elegidos a mano, los colores son semanticos.
2. **Significado de cada estado** explicado en parrafos completos. El HTML era para presentar; este manual es para *entender*.
3. **Anclaje al codigo real del INTISAT.** Cuando una FSM ya existe en el repositorio, lo marcamos. Cuando todavia es propuesta, tambien.

Como leerlo. Lo podes leer secuencialmente (capitulos 1–3 te dan el marco; 4–9 cada subsistema; 10 el roadmap) o usarlo de referencia abriendo solo el capitulo del subsistema que te toca.

Índice general

1. Que es una maquina de estados y por que importa	1
1.1. La definicion formal y la version honesta	1
1.2. Por que los CubeSats no usan codigo plano	1
1.3. La gran intuicion: en cualquier momento, un solo estado	2
2. Los 3 niveles de jerarquia	3
2.1. Nivel 1 — la FSM de MISION	3
2.2. Nivel 2 — la FSM de cada SUBSISTEMA	4
2.3. Nivel 3 — componentes	4
3. La FSM de MISION: el satellite completo	5
3.1. LAUNCH — de la base de lanzamiento al deployer	5
3.2. DETUMBLE — frenar el spin de la eyeccion	6
3.3. COMMISSIONING — la primera semana en orbita	6
3.4. NOMINAL — el modo “default”	7
3.5. PAYLOAD_OP — el experimento	7
3.6. COMMS_PASS — la ventana de comunicacion	7
3.7. ECLIPSE — el lado oscuro de la orbita	8
3.8. SAFE — el modo de proteccion	8
3.9. EOL — end of life	8
4. La matriz modos x subsistemas (la “rosetta”)	9
4.1. Como leerla	9
4.2. Por que esta matriz es importante	10
4.3. Las reglas de coherencia	10
5. OBC — el cerebro central	11
5.1. Que hace el OBC	11
5.2. Los 5 estados, en detalle	12
5.3. La regla del watchdog	13
6. EPS — el corazon energetico	14
6.1. Por que EPS es el subsistema mas critico	14
6.2. Los 7 estados, en detalle	15
6.3. La estimacion del SoC	16
7. ADCS — a donde apunta el satellite	17
7.1. Por que SUN_POINTING es el default	17
7.2. Los 8 estados	18
7.3. Hardware tipico de un ADCS CubeSat	18

8. COMMS — la voz del satellite	20
8.1. Las dos bandas tipicas	20
8.2. Los 7 estados	21
8.3. Por que BEACON es tan importante	21
9. PAYLOAD — el caso INTISAT	22
9.1. Por que este es el subsistema mas avanzado	22
9.2. Los 9 estados, en detalle	23
9.3. El camino feliz vs los caminos de error	24
10.Comparacion contra el INTISAT actual	25
10.1. Estado del arte por subsistema	25
10.2. Plan sugerido	25
A. Tabla formal de transiciones (extracto)	27
B. Referencias para validar	28

Capítulo 1

Que es una maquina de estados y por que importa

1.1. La definicion formal y la version honesta

Una **maquina de estados finitos** (FSM, *Finite State Machine*) es una tupla $(S, \Sigma, \delta, s_0, F)$ donde:

- S es un conjunto finito de *estados*.
- Σ es un conjunto finito de *eventos* (triggers).
- $\delta : S \times \Sigma \rightarrow S$ es la funcion de transicion.
- $s_0 \in S$ es el estado inicial.
- $F \subseteq S$ son estados terminales (opcional).

Esta es la version del libro de teoria de computacion. La version honesta, para alguien que esta programando un satellite, es:

Intuicion

Una FSM te obliga a escribir, ANTES de codear, una lista de:

1. Todos los modos en los que puede estar mi sistema.
2. Que eventos pueden hacer que cambie de modo.
3. Cual es la transicion exacta para cada par (modo, evento).

Si no podes responder estas 3 preguntas, no sabes que hace tu sistema realmente. Y si no lo sabes en Tierra, vas a tener un satellite que hace cosas que no entiendes en orbita.

1.2. Por que los CubeSats no usan codigo plano

En un servidor web, si el codigo hace algo raro, te conectas por SSH, mirás los logs, corrés un `strace` y arreglás. En un CubeSat no podes hacer nada de eso. Tenes:

- **Ventanas de comunicacion** de 5–10 minutos por orbita, cada ~ 90 min.
- **Ancho de banda** que va de 1.2 kbps (UHF) a 100 kbps (S-Band en el mejor caso).

- **Sin acceso fisico.** Si algo se bloquea, esperas el proximo watchdog reset (puede ser horas).
- **Decisiones autonomas.** El satellite tiene que poder decidir solo cuando entrar en SAFE, cuando hacer downlink, cuando apagar el payload.

Cuidado

Si tu codigo del satellite tiene 50 variables booleanas tipo `is_charging`, `is_pointing`, `can_transmit`, `has_data_ready`... tenes una FSM *implicita* con 2^{50} estados posibles. La mayoria de esos estados son invalidos o catastroficos, pero no puedes razonar sobre ellos.

Hacer la FSM explicita con ~ 10 estados bien definidos no es “hacer mas trabajo”; es *descubrir* cuales de los 2^{50} estados realmente queres que existan.

1.3. La gran intuicion: en cualquier momento, un solo estado

La regla de oro de una FSM es: **en cualquier instante, el sistema esta en EXACTAMENTE un estado.** No dos. No medio. Uno.

Esto suena trivial pero es lo mas duro de respetar cuando uno arranca. La tentacion natural es decir “bueno, estoy cargando la bateria *y* apuntando al sol *y* esperando un comando”. La FSM te obliga a decir: “estoy en NOMINAL, lo cual implica que estoy cargando, apuntando y esperando, pero el unico estado es NOMINAL”.

Esa disciplina convierte un sistema que “a veces no funciona y no sabemos por que” en un sistema donde puedes preguntar “en que estado estabas?” y la respuesta es univoca.

Capítulo 2

Los 3 niveles de jerarquia

Un CubeSat no es *una* maquina de estados. Es una jerarquia de tres niveles. Confundirlos es el error mas comun.

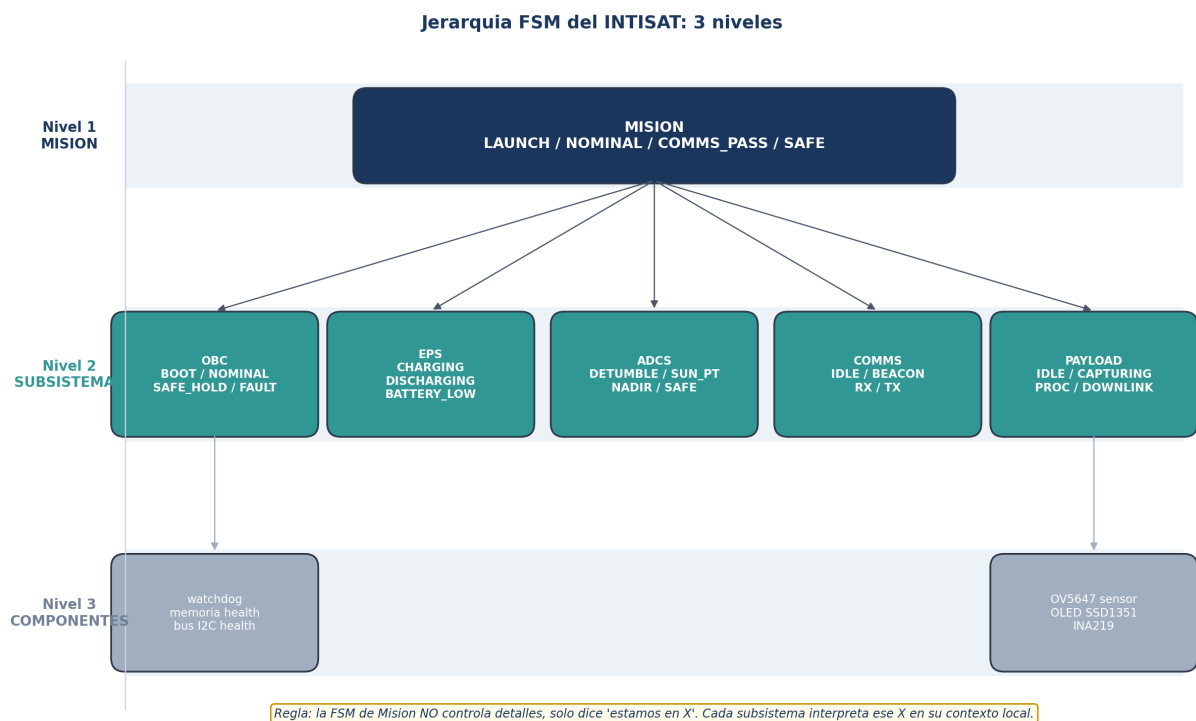


Figura 2.1: Jerarquia de FSMs en el INTISAT. La FSM de mision (nivel 1) no controla detalles; cada subsistema (nivel 2) tiene su propia FSM, e internamente cada componente (nivel 3) tambien.

2.1. Nivel 1 — la FSM de MISION

Es lo que ve el operador de tierra. Son 8 modos a alto nivel: LAUNCH, DETUMBLE, COMMISSIONING, NOMINAL, PAYLOAD_OP, COMMS_PASS, ECLIPSE, SAFE, EOL. La FSM de mision **NO** mira los pines del CM5, no sabe que el OLED esta encendido, no sabe el voltaje del shunt. Solo sabe: “estamos en NOMINAL” o “estamos en SAFE”.

2.2. Nivel 2 — la FSM de cada SUBSISTEMA

Cada subsistema (OBC, EPS, ADCS, COMMS, PAYLOAD) tiene su propia FSM con sus propios estados internos. Cuando la mision esta en NOMINAL, cada subsistema interpreta “NOMINAL” en su contexto local:

- EPS en NOMINAL = “estoy CHARGING”.
- ADCS en NOMINAL = “estoy en SUN_POINTING”.
- PAYLOAD en NOMINAL = “estoy en IDLE, listo para CMD_START_CAPTURE”.

La traduccion entre “modo de mision” y “estado interno de cada subsistema” la define la **matriz modos × subsistemas** (capitulo 4).

2.3. Nivel 3 — componentes

Bajo cada subsistema hay componentes con su propia mini-FSM:

- En el OBC: el watchdog timer, el chequeo de memoria, el bus I2C.
- En el PAYLOAD: el sensor OV5647 (apagado / configurado / capturando), el OLED SSD1351 (apagado / mostrando estado), el INA219 (idle / midiendo).

Intuicion

La regla: cada nivel coordina al de abajo pero no se mete en detalles. La FSM de mision NO le dice al ADCS “encende el reaction wheel X”; le dice “estamos en PAYLOAD_OP” y el ADCS sabe que en PAYLOAD_OP tiene que estar en NADIR_POINTING, y para eso comanda a sus componentes internos.

Capítulo 3

La FSM de MISION: el satelite completo

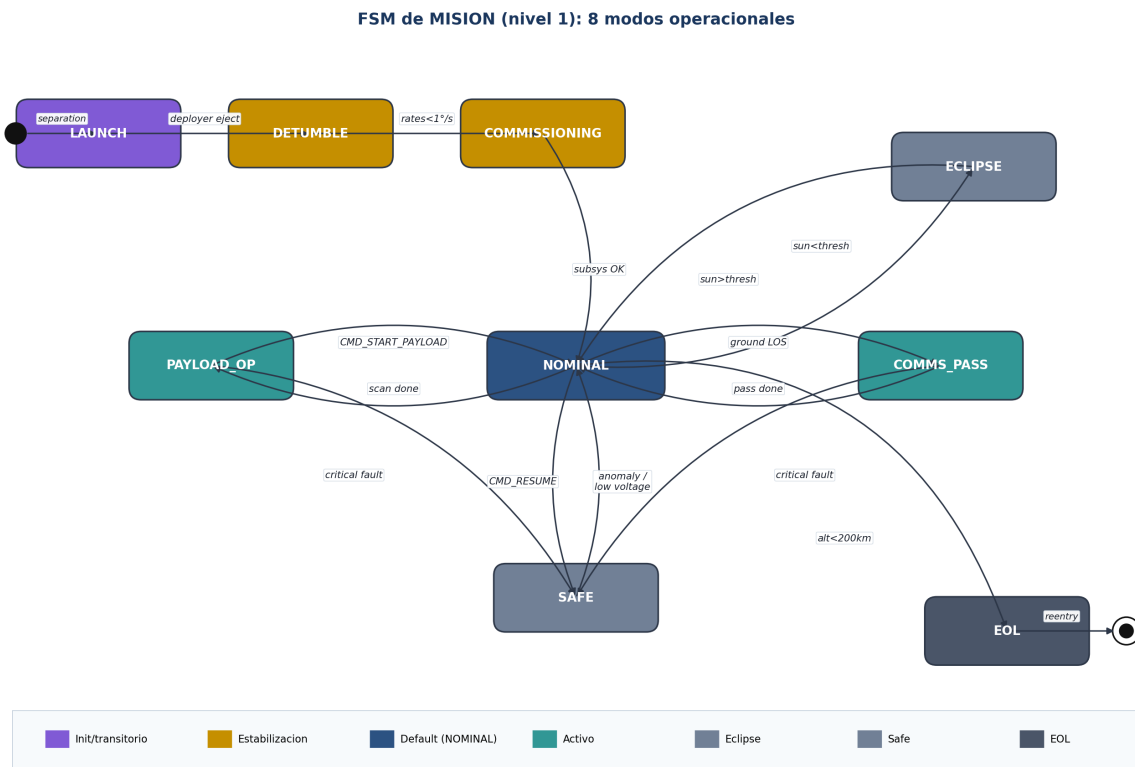


Figura 3.1: FSM de mision. 8 modos operacionales del INTISAT, con transiciones desde la separacion del deployer hasta el end-of-life.

A continuacion describimos cada uno de los 8 modos. Para cada uno indicamos: **(a)** cuando se entra, **(b)** que hace el satelite en ese modo, **(c)** cuando se sale.

3.1. LAUNCH — de la base de lanzamiento al deployer

(a) Entrada: desde el momento que el satelite esta en el deployer del cohete hasta unos minutos despues de la eyeccion. Tipicamente entre T-30 min y T+45 min, donde T es la hora de lanzamiento.

(b) Que hace: *nada*. Por norma de seguridad de los deployers (NanoRacks, ISIPOD, etc.),

el CubeSat debe estar ELECTRICAMENTE INERTE hasta que el switch de despliegue se cierra. Esto implica:

- Bateria conectada pero rails de potencia abiertos.
- OBC en estado BOOT (no NOMINAL).
- Radio apagado (no se puede transmitir desde dentro del cohete).
- ADCS apagado (los actuadores no deben moverse).

(c) **Salida:** cuando el switch de despliegue (RBF + deployment switches) se cierra. Típicamente ~ 30 minutos después de la eyección, según ECSS-E-ST-70-11C.

3.2. DETUMBLE — frenar el spin de la eyección

(a) **Entrada:** inmediatamente después de LAUNCH.

(b) **Que hace:** el deployer eyecta el CubeSat con un *spin residual* de hasta $5\text{--}10^\circ/\text{s}$. Hay que frenarlo para poder hacer cualquier cosa útil (apuntar, comunicar, generar energía). El algoritmo clásico es *B-dot*:

$$\vec{m}_{cmd} = -k \cdot \frac{d\vec{B}}{dt} \quad (3.1)$$

donde $\vec{B}$ es el campo magnético terrestre medido por el magnetómetro y $\vec{m}_{cmd}$ es el dipolo magnético comandado a los magnetorquers. El efecto es disipar energía rotacional sin necesidad de conocer la actitud absoluta. Lleva entre 6 y 48 horas.

(c) **Salida:** cuando $\|\vec{\omega}\| < 1^\circ/\text{s}$ en los 3 ejes (criterio típico).

Detalle técnico

B-dot funciona porque la derivada del campo magnético en frame body es proporcional al spin. Es robusto: no requiere knowledge de actitud, solo magnetómetro funcional. Es el algoritmo de “última esperanza” — si todo lo demás falla, vuelves a B-dot.

3.3. COMMISSIONING — la primera semana en órbita

(a) **Entrada:** cuando termina DETUMBLE.

(b) **Que hace:** chequeo sistemático de cada subsistema. Esta fase típicamente dura entre 1 y 2 semanas (no horas como muchos piensan). Incluye:

- Verificar que todos los rails de potencia entregan voltaje correcto.
- Calibrar magnetómetro y sensores de sol.
- Validar el link RF con la estación de tierra (downlink + uplink).
- Probar cada subsistema en aislamiento, después en conjunto.
- Verificar que la temperatura de los componentes está en rango.

(c) **Salida:** cuando el equipo de operaciones declara “commissioning complete”. Es una decisión humana.

3.4. NOMINAL — el modo “default”

(a) **Entrada:** terminado COMMISSIONING, o desde COMMS_PASS / PAYLOAD_OP / SAFE.

(b) **Que hace:** es el modo “*quiescente*”. El satellite:

- Apunta paneles al sol (ADCS: SUN_POINTING).
- Carga bateria (EPS: CHARGING).
- OBC corriendo nominal, escuchando uplink.
- Radio en IDLE (RX latente, BEACON cada 30 s).
- Payload OFF o IDLE.

Es el modo donde el INTISAT pasa ~80 % del tiempo.

(c) **Salida:** hacia PAYLOAD_OP (comando), COMMS_PASS (entra en LOS con la estacion), ECLIPSE (sol bajo), SAFE (anomalia), EOL (altura final).

3.5. PAYLOAD_OP — el experimento

(a) **Entrada:** comando CMD_START_PAYLOAD desde tierra.

(b) **Que hace:** es el “why we’re here”. Para el INTISAT:

1. ADCS apunta a nadir (NADIR_POINTING) durante ~10 min.
2. Payload despierta del IDLE: PAYLOAD: CAPTURING.
3. Captura 25 frames con OV5647 + mascara lensless.
4. Procesa con la red neuronal (Real-ESRGAN o similar).
5. Exporta a almacenamiento.
6. Vuelve a IDLE y devuelve el bus a NOMINAL.

(c) **Salida:** scan completo (vuelve a NOMINAL), o fault critico (va a SAFE).

Ya implementado en INTISAT

El payload del INTISAT YA TIENE esta secuencia implementada como FSM en `cubesat/commands.py:STATE_*`. La estudiamos en detalle en el capitulo 9.

3.6. COMMS_PASS — la ventana de comunicacion

(a) **Entrada:** cuando entra LOS (*Line Of Sight*) con la estacion de tierra. Se predice con TLE (Two-Line Element).

(b) **Que hace:**

- ADCS apunta antena al ground station (TARGET_POINTING).
- COMMS pasa de IDLE a TX (downlink) o RX (uplink).
- Payload puede estar haciendo DOWNLINK si quedo data de scans previos.

(c) **Salida:** cuando se pierde LOS. Tipicamente 5–10 min por pase.

3.7. ECLIPSE — el lado oscuro de la orbita

(a) **Entrada:** cuando el sensor de sol detecta $<5\%$ del flujo nominal.

(b) **Que hace:**

- EPS pasa a DISCHARGING (sin generacion solar).
- ADCS pasa a INERTIAL_HOLD (no hay vector solar para apuntar).
- OBC opcionalmente entra en MAINTENANCE (limpieza de logs, defrag).
- Payload normalmente OFF (a menos que se programe un scan nocturno).

(c) **Salida:** cuando vuelve el sol. Tipicamente 35 min en orbita baja.

3.8. SAFE — el modo de proteccion

(a) **Entrada:** CUALQUIER anomalia critica. Ejemplos:

- EPS reporta BATTERY_CRITICAL.
- Watchdog del OBC se dispara 3 veces.
- Temperatura fuera de rango.
- Comando ground CMD_SAFE explicito.

(b) **Que hace:** apaga TODO lo no esencial. Solo quedan:

- OBC en SAFE_HOLD (consumo ~ 1 W).
- COMMS en BEACON (transmite telemetria minima).
- ADCS en DETUMBLE (proteccion contra spin).

NO acepta comandos no-criticos. Solo CMD_RESUME y comandos de telemetria.

(c) **Salida:** comando CMD_RESUME desde tierra, DESPUES de que el equipo de operaciones haya verificado que la causa de la anomalia esta resuelta.

Cuidado

SAFE no es “modo error”. Es “modo conservador hasta que humanos decidan”. Un CubeSat puede pasar dias en SAFE sin problema. Salir de SAFE prematuramente es la causa numero uno de perdida total de mision.

3.9. EOL — end of life

(a) **Entrada:** cuando la altitud cae por debajo de un umbral (tipicamente 200 km) o cuando la mision se declara terminada por otra razon.

(b) **Que hace:** preparacion para reentrada:

- Borrado seguro de datos sensibles.
- Beacon final con telemetria de fin de vida.
- ADCS off (los magnetorquers no aceleran la reentrada).

(c) **Salida:** la reentrada atmosferica completa la mision.

Capítulo 4

La matriz modos x subsistemas (la “rosetta”)

Matriz modos x subsistemas (rosetta)					
	OBC	EPS	ADCS	COMMS	PAYLOAD
LAUNCH	BOOT	NOMINAL	OFF	IDLE	OFF
DETUMBLE	NOMINAL	DISCHARG	DETUMBLE	BEACON	OFF
COMMISSIONING	NOMINAL	CHARGING	SUN_PT	BEACON	OFF
NOMINAL	NOMINAL	CHARGING	SUN_PT	IDLE	IDLE
PAYLOAD_OP	NOMINAL	DISCHARG	NADIR_PT	IDLE	CAPTURE
COMMS_PASS	NOMINAL	DISCHARG	TARGET	TX/RX	DOWNLNK
ECLIPSE	MAINT	DISCHARG	INERTIAL	IDLE	IDLE
SAFE	SAFE_HLD	CRITICAL	DETUMBLE	BEACON	SAFE
EOL	NOMINAL	DISCHARG	OFF	BEACON	OFF

OFF/IDLE

Normal

Activo

Safe/Fault

Figura 4.1: Matriz que traduce cada modo de mision al estado interno de cada subsistema. Es el contrato entre los equipos.

4.1. Como leerla

Buscas una fila (el modo de mision) y una columna (el subsistema). La celda te dice en que estado interno tiene que estar ese subsistema cuando la mision esta en ese modo.

Ejemplo: cuando la mision esta en PAYLOAD_OP, el ADCS tiene que estar en NADIR_POINTING (no en SUN_POINTING), el EPS en DISCHARGING (consumimos mas de lo que generamos durante el scan), y el PAYLOAD en CAPTURE.

4.2. Por que esta matriz es importante

Es el documento que **evita las discusiones entre equipos**. Si el lider de ADCS y el lider de Payload no se ponen de acuerdo sobre si NADIR_POINTING o TARGET_POINTING corresponde a PAYLOAD_OP, la matriz lo decide. Una vez firmada, es el contrato.

4.3. Las reglas de coherencia

Hay reglas que la matriz debe cumplir para ser consistente:

1. En SAFE, todos los subsistemas deben estar en su estado *mas conservador*.
2. En cualquier modo, EPS debe poder sostener el consumo. Si la suma de consumos de cada subsistema en ese modo $>$ la potencia generada + la bateria disponible, ese modo es invalido.
3. Los modos de los subsistemas deben ser alcanzables desde su estado anterior (no podemos saltar de IDLE a CAPTURING sin pasar por BOOT en el caso del payload, por ejemplo).

Intuicion

La matriz se llena *a mano*, en reunion con todos los lideres tecnicos. Hay 9 modos x 5 subsistemas = 45 celdas. Cada celda lleva una conversacion: “en COMMS_PASS, donde tiene que estar el payload?” — “en IDLE” — “y si tenemos data pendiente?” — “ah, entonces puede estar en DOWNLINK”. Esa conversacion la haces una vez, la escribis, y nunca mas vuelve.

Capítulo 5

OBC — el cerebro central

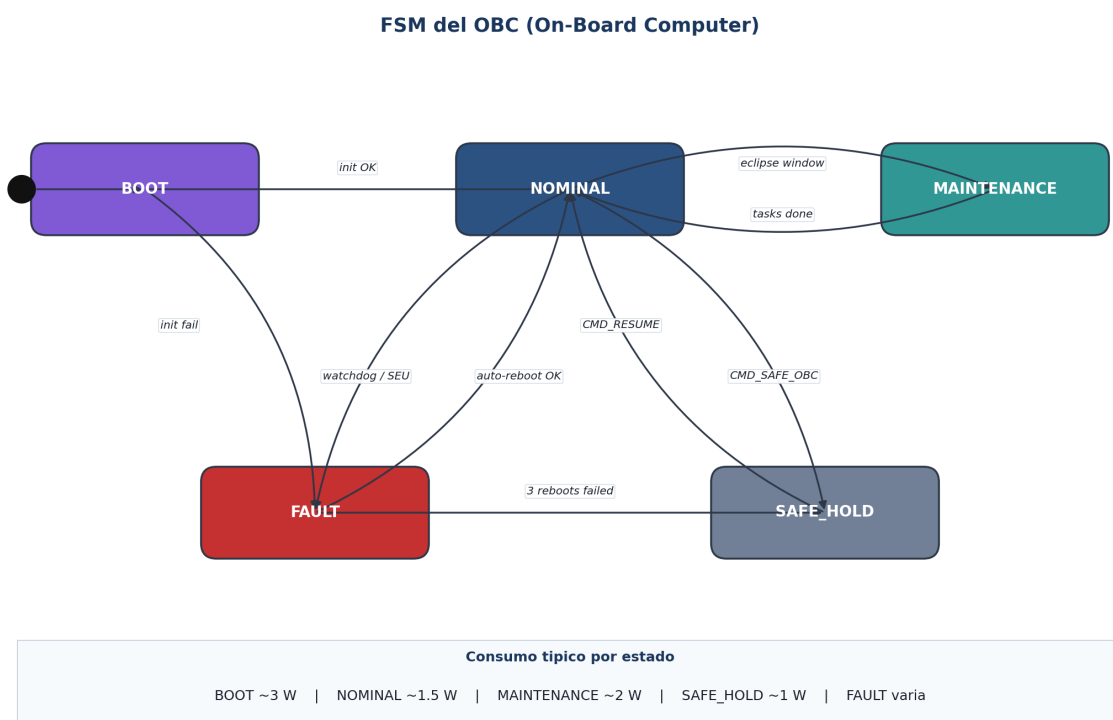


Figura 5.1: FSM del On-Board Computer. 5 estados, con consumo típico por estado en la banda inferior.

5.1. Que hace el OBC

El OBC (On-Board Computer) es el *coordinador* del satélite. Es la pieza que:

- Mantiene el reloj de misión (epoch + uptime).
- Decide en qué modo de misión está el satélite.
- Recibe comandos del COMMS y los enruta al subsistema correcto.
- Recoge telemetría de todos los subsistemas (vía I2C, SPI, CAN — depende del diseño) y la prepara para downlink.
- Aplica reglas FDIR (*Fault Detection, Isolation and Recovery*).

En el INTISAT, el OBC NO es el mismo que el CM5 del payload. El OBC sera un micro-controlador robusto (LPC1768, STM32F4 rad-tolerant, o similar) que conversa con el CM5 por I2C.

5.2. Los 5 estados, en detalle

BOOT

Que pasa: kernel arranca, servicios systemd levantan, chequeo de memoria, chequeo de bus I2C, lectura de configuracion de NVRAM. Dura ~ 15 segundos. Consumo: ~ 3 W (transitorio).

Salidas: NOMINAL si todo OK; FAULT si init fallido (memoria corrupta, peripheral no responde, etc.).

NOMINAL

Que pasa: el OBC corre su loop principal: lee telemetria, verifica reglas FDIR, atiende comandos de COMMS, escribe logs en ring buffer. Consumo: ~ 1.5 W.

Salidas: MAINTENANCE (entrada programada en eclipse), FAULT (watchdog timeout o SEU detectado), SAFE_HOLD (comando ground).

Detalle tecnico

SEU (Single Event Upset) es un bit-flip causado por particulas cosmicas. En LEO ocurre uno cada $\sim$ dia por GB de memoria. Detectarlo requiere ECC en RAM o checksums en flash. Si el OBC no tiene ECC, hay que correr `memtest` periodico (que es lento) o disenar para que el watchdog reset sea “barato” (boot < 30 s).

MAINTENANCE

Que pasa: tareas de housekeeping. Limpieza de logs viejos, defrag de la SD/eMMC, escritura de checkpoints, actualizacion de TLEs si vino comando de ground. Consumo: ~ 2 W.

Tipicamente se programa para el eclipse, donde no hay otra actividad util (no se puede hacer payload por que el ADCS no apunta nada).

Salida: NOMINAL cuando termine la lista de tareas.

SAFE_HOLD

Que pasa: OBC en consumo minimo, solo acepta una lista chiquita de comandos:

- `CMD_TLM_DUMP` (volcar telemetria).
- `CMD_RESUME` (volver a NOMINAL).
- `CMD_REBOOT` (reset forzado).

Consumo: ~ 1 W. Es el estado del OBC cuando la mision esta en SAFE.

Salida: NOMINAL con `CMD_RESUME`.

FAULT

Que pasa: el watchdog se disparo o hay un fault. El OBC intenta auto-reset. Si en 3 intentos no vuelve a NOMINAL, escala a SAFE_HOLD para evitar boot-loop infinito.

Salidas: NOMINAL (auto-reset exitoso), SAFE_HOLD (3 resets fallidos).

5.3. La regla del watchdog

El watchdog timer es un hardware dedicado (no se puede deshabilitar desde software facilmente). Cuenta hacia abajo. Si no se “patea” (re-arma) en X segundos, resetea el procesador. Esto protege contra hang del software.

En el INTISAT, el watchdog tipico es de 30–60 s. El loop principal del OBC debe patearlo cada ~ 5 s. Si tarda mas de 30 s, asumimos que el OBC esta colgado y reseteamos.

Cuidado

NUNCA deshabilites el watchdog “temporalmente para debug”. Es la causa numero dos de perdida de mision (numero uno: salir de SAFE prematuramente). Si necesitas debug local en banco, usa una configuracion separada con watchdog mas largo.

Capítulo 6

EPS — el corazon energetico

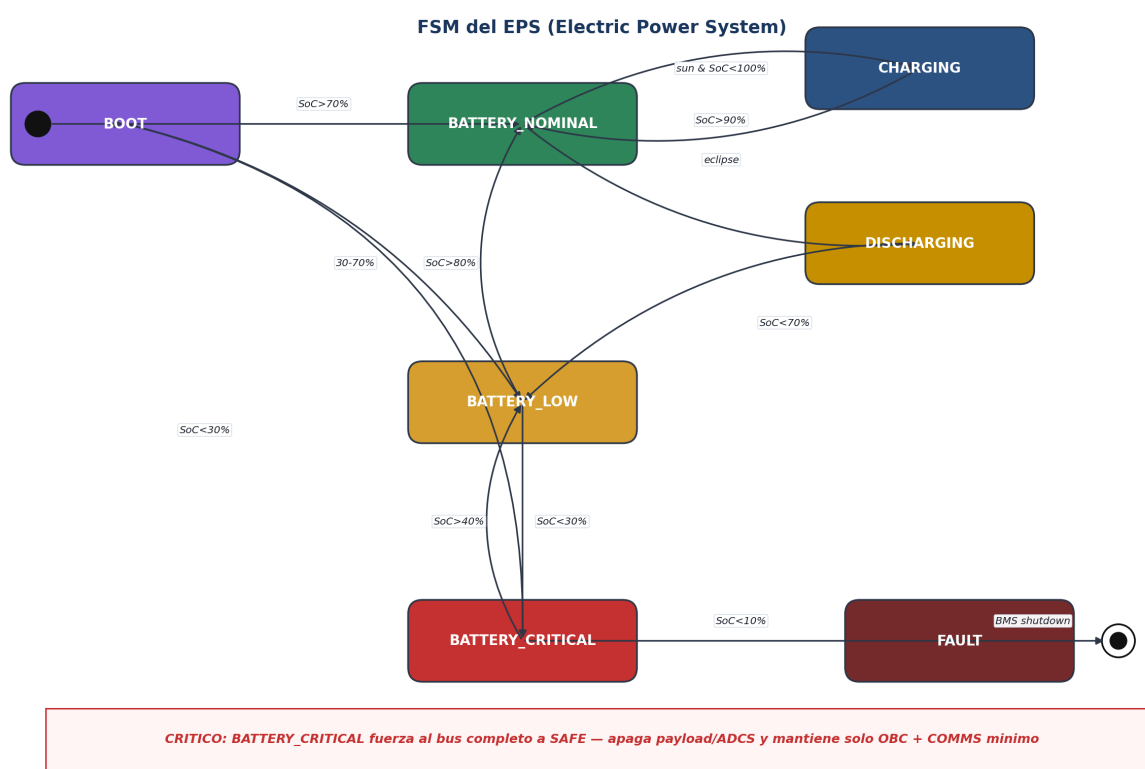


Figura 6.1: FSM del EPS. Los estados de bateria forman una cadena que va de NOMINAL hasta CRITICAL, con la posibilidad de FAULT irreuperable si SoC cae por debajo de 10 %.

6.1. Por que EPS es el subsistema mas critico

Porque si EPS falla, todo lo demas se apaga, ni siquiera tenes telemetria para diagnosticar el fallo. EPS tiene autoridad para:

- Forzar al OBC a SAFE_HOLD si SoC < umbral.
- Apagar rails de potencia no esenciales.
- Reactivar rails despues de un fault transitorio.
- Reportar telemetria de salud propia incluso si OBC esta caido.

En arquitecturas profesionales (e.g., GomSpace NanoPower P31u), el EPS tiene su propio microcontrolador (TI MSP430 tipico) totalmente independiente del OBC.

6.2. Los 7 estados, en detalle

BOOT

Al power-on, el EPS lee SoC de la bateria (via gauge dedicado, e.g., LTC2942) y decide en cual de los 3 estados de bateria arrancar:

- $\text{SoC} > 70\% \rightarrow \text{BATTERY_NOMINAL}$.
- $30\% < \text{SoC} < 70\% \rightarrow \text{BATTERY_LOW}$ (no enciende payload).
- $\text{SoC} < 30\% \rightarrow \text{BATTERY_CRITICAL}$ (solo OBC + COMMS).

BATTERY_NOMINAL

Estado feliz. Todos los rails alimentados. Es el “IDLE” del EPS. Subestados: CHARGING (si vemos sol y $\text{SoC} < 100\%$) o DISCHARGING (si estamos en eclipse).

CHARGING / DISCHARGING

No son “modos distintos” del EPS desde el punto de vista del satellite; son flags internos. El EPS sigue siendo BATTERY_NOMINAL externamente. Internamente es importante para el MPPT y para predecir cuanta autonomia queda.

BATTERY_LOW

Cuando SoC cae por debajo de 70 % en DISCHARGING (eclipse largo o consumo alto). El EPS **notifica al OBC** para que considere suspender operaciones no esenciales (e.g., no arrancar un payload scan nuevo).

Para volver a NOMINAL hace falta $\text{SoC} > 80\%$ (hysteresis para evitar chattering entre estados).

BATTERY_CRITICAL

$\text{SoC} < 30\%$. Esto es serio. El EPS:

1. Manda evento de SAFE al OBC (que pasa todo el satellite a SAFE).
2. Apaga rails de payload, ADCS no esencial, etc.
3. Mantiene solo OBC en SAFE_HOLD + COMMS en BEACON.

Para salir de CRITICAL hace falta $\text{SoC} > 40\%$ (recuperacion lenta con luz solar).

FAULT

$\text{SoC} < 10\%$. El BMS (Battery Management System) corta el bus principal para proteger las celdas de over-discharge. Esto es **normalmente irrecuperable** — el satellite queda sin telemetria. Es un evento “mission end”.

Cuidado

Las celdas Li-ion sufren daño permanente si bajan de 2.5 V/celda. El BMS corta a ~ 2.7 V/celda como margen. Una vez que cae mas bajo, aunque vuelva la luz solar, la celda puede no aceptar carga o aceptar mucho menos de la capacidad original. Por eso **CRITICAL** $\rightarrow$ **FAULT** es a 10 % (no a 5 %): para tener margen de la latencia de respuesta del BMS.

6.3. La estimacion del SoC

El SoC (State of Charge) NO se mide directamente. Se estima. Hay tres metodos comunes:

1. **Coulomb counting:** integrar la corriente entrante y saliente. Precisa pero acumula error (drift) sin recalibracion.
2. **Open-Circuit Voltage (OCV):** medir voltaje sin carga. Preciso pero requiere periodos de no-current (eclipse).
3. **Modelo Kalman:** combina coulomb counting con OCV. Es lo que usan los gauges modernos (LTC2942, MAX17048).

Capítulo 7

ADCS — a donde apunta el satellite

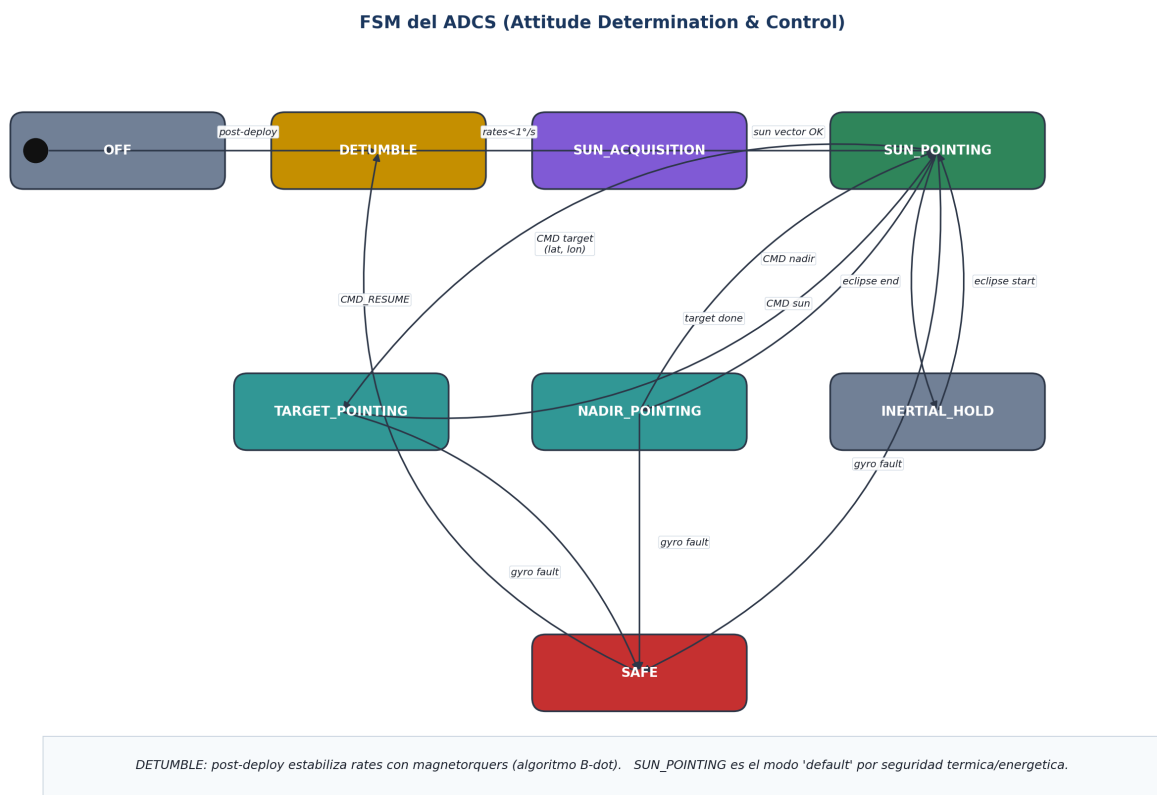


Figura 7.1: FSM del ADCS. SUN_POINTING es el modo “default” (por seguridad termica). Cualquier modo activo (NADIR/TARGET) puede volver via SUN_POINTING.

7.1. Por que SUN_POINTING es el default

Toda FSM tiene un “estado seguro pero util”. Para el ADCS, ese es SUN_POINTING. Razones:

1. **Energia:** si los paneles apuntan al sol, generamos corriente. Si apuntamos a otro lado, descargamos bateria.
2. **Termica:** apuntar al sol mantiene una cara caliente y otra fria, en lugar de barrer todo el satellite por toda la expansion termica.

3. **Conocido:** el vector solar es facil de calcular con Kepler. El vector nadir requiere conocer altitud y attitude. El vector target requiere conocer la posicion del ground station *y* la del satellite simultaneamente.

7.2. Los 8 estados

OFF

ADCS apagado. Estado en LAUNCH (antes de despliegue) y en EOL.

DETUMBLE

Activacion del algoritmo B-dot. Magnetorquers comandados a frenar el spin residual. Sin reaction wheels en esta fase (los wheels requieren actitud conocida; aqui no la tenemos).

SUN_ACQUISITION

Una vez detumbled, buscar el vector solar. Si los sensores de sol no detectan flujo, el satellite hace un spin lento ($\sim 2^\circ/\text{s}$) para barrer el cielo.

SUN_POINTING

El modo default. Reaction wheels (o magnetorquers) mantienen los paneles solares ortogonales al vector solar. Precision tipica: $\pm 2-5^\circ$.

NADIR_POINTING

Para PAYLOAD_OP (microscopia mirando hacia Tierra). Apuntar el eje +Z hacia nadir. Precision requerida depende del payload; para microscopia FPM lensless el requirement es relativamente laxo ($\pm 1-2^\circ$).

TARGET_POINTING

Para COMMS_PASS, apuntar la antena hacia el ground station. Esto requiere TLE actualizado y conocer la posicion del ground station. Precision tipica $\pm 0.5^\circ$ para antena directiva.

INERTIAL_HOLD

Durante eclipse, mantener actitud actual sin gastar energia en correcciones agresivas. Wheels se mantienen en momentum bajo.

SAFE

Cualquier fault del giroscopo o magnetometro principal. Vuelve a DETUMBLE para recuperar baseline.

7.3. Hardware tipico de un ADCS CubeSat

- **Magnetometro:** HMC5883L, MMC5983MA. 3 ejes.
- **Giroscopo:** L3G4200D, BMI088. 3 ejes.
- **Sensor de sol:** fotodiodos en cada cara (6 caras) o *sun sensors* (NanoSSOC-D60, etc.).
- **Magnetorquers:** bobinas en X, Y, Z. Tipicamente $0.1-0.2 \text{ Am}^2$.

- **Reaction wheels:** opcional. Para precision $< 1^\circ$.

Capítulo 8

COMMS — la voz del satélite

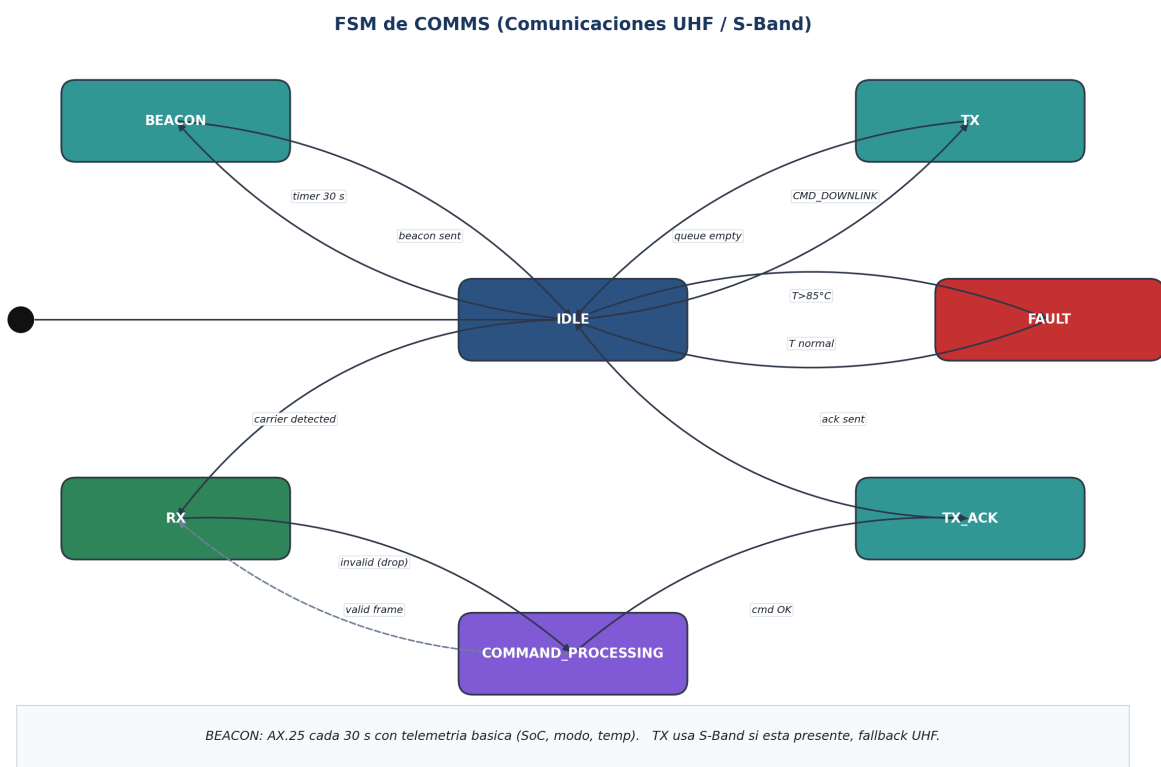


Figura 8.1: FSM de comunicaciones. IDLE es el hub central; BEACON, RX, TX y los caminos de comando giran alrededor.

8.1. Las dos bandas típicas

- **UHF (430–440 MHz):** amateur radio band, sin licencia comercial. Throughput 1.2–9.6 kbps. Antena omnidireccional. Usado para comandos y beacon.
- **S-Band (2200–2300 MHz):** requiere licencia. Throughput hasta 1 Mbps con antena direccional. Usado para downlink de datos (imagenes, video).

INTISAT comienza con solo UHF. S-Band se agregaria en una version futura si el volumen de datos lo justifica.

8.2. Los 7 estados

IDLE

Radio encendido pero sin actividad. Receiver escuchando, transmitter apagado. Consumo bajo. Es el “hub” de la FSM de COMMS.

BEACON

Cada 30 s, transmite paquete corto AX.25 con telemetria minima: SoC, modo de mision, temperatura, contador de uptime. Permite que cualquier radioaficionado en el mundo confirme que el satellite esta vivo.

RX

Carrier detectado. Receptor demodula y entrega paquetes al processor para decodificacion.

COMMAND_PROCESSING

Frame valido recibido. Verifica CRC, autenticacion (HMAC), y secuencia anti-replay. Si pasa, ejecuta. Si falla, descarta y vuelve a RX.

TX_ACK

Despues de ejecutar un comando, envia ack con resultado.

TX

Comando explicito CMD_DOWNLINK dispara una sesion de transmision. Vacía la queue de telemetria/datos. Si tiene S-Band, usa S-Band; fallback UHF.

FAULT

Temperatura del PA (Power Amplifier) sobre 85 °C, o SWR muy alto (antena rota o conector flojo). Se apaga TX hasta que la temperatura normalice.

8.3. Por que BEACON es tan importante

Un satellite que beaconeas es un satellite vivo. Aunque el resto este caido, mientras el beacon llegue a tierra:

- Sabemos donde esta orbitando (refinamos TLE con doppler).
- Sabemos que la bateria no esta muerta.
- Tenemos un canal para diagnosticar.

Diseñar BEACON como hardware con un timer dedicado (no dependiente del OBC) es una buena practica defensiva.

Capítulo 9

PAYLOAD — el caso INTISAT

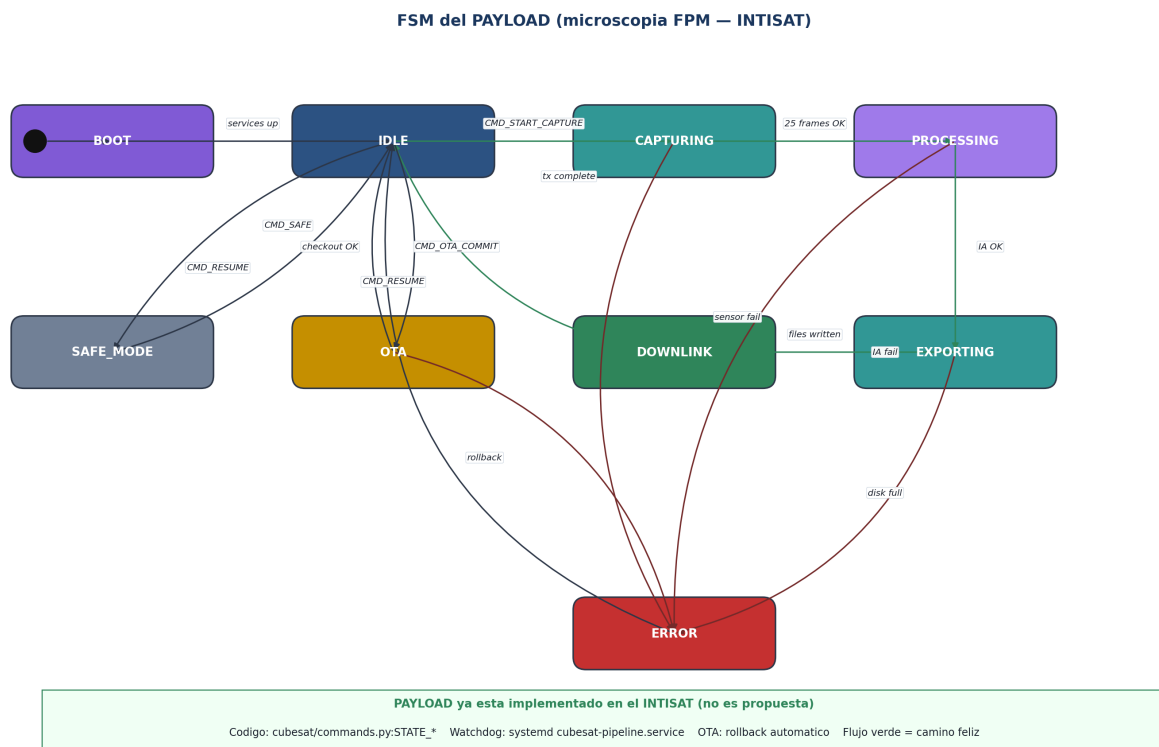


Figura 9.1: FSM del payload de microscopia FPM del INTISAT. El camino verde es el “happy path”. Errores caen a ERROR central, recuperable via `CMD_RESUME`.

9.1. Por que este es el subsistema mas avanzado

Porque ya esta implementado. No es propuesta. Esta corriendo en el laboratorio, en una Raspberry Pi 5, con codigo en `cubesat/`.

Ya implementado en INTISAT

La FSM del payload existe en codigo, no solo en este documento. Los estados estan en `cubesat/commands.py:STATE_*`. Las transiciones estan en `cubesat/daemon.py`. El watchdog esta en `cubesat-pipeline.service` (`systemd`).

9.2. Los 9 estados, en detalle

BOOT

Inicializacion al power-on:

1. Levantar servicios systemd (`cubesat-pipeline.service`).
2. Cargar configuracion de `config.yaml`.
3. Inicializar OV5647 (libcamera `rpicam-hello` para sanity check).
4. Inicializar OLED SSD1351 (`luma.oled`).
5. Inicializar INA219 (`i2c-tools`, address 0x40).
6. Conectar a I2C slave con OBC (`pigpio bsc`, address 0x42).

Tipicamente ~ 30 segundos. Consumo: ~ 5 W transitorio.

Salidas: IDLE si todo OK; ERROR si algun init fallido.

IDLE

Pipeline corriendo pero sin captura. OLED muestra estado. INA219 sampling cada 5 s. Esperando comandos.

Consumo ~ 2 W.

CAPTURING

CMD_START_CAPTURE recibido. Secuencia:

1. Recibir parametros (ISO, exposure, num_frames).
2. Activar OLED para iluminacion del DUT (Device Under Test).
3. Cambiar mascara (si aplica, mecanismo MEMS o por filtro movil).
4. Capturar N frames con OV5647.
5. Guardar raws en `/scans/raw/{timestamp}/`.

Consumo ~ 6 W (sensor + OLED + procesamiento ligero).

Salidas: PROCESSING al completar N frames; ERROR si sensor falla.

PROCESSING

Inferencia con la red. Tres pasos:

1. Stack: alinear los N frames (registracion).
2. Reconstruccion FPM o super-resolucion (Real-ESRGAN-x4).
3. Guardar resultado en `/scans/processed/{ts}/`.

Consumo ~ 8 W pico (CPU saturada, sin GPU acelerada en RPi 5 por ahora). Tiempo: ~ 60 – 90 s por scan de 25 frames.

Salidas: EXPORTING si IA OK; ERROR si inferencia falla (e.g., out-of-memory).

EXPORTING

Empaquetado para downlink:

1. Comprimir imagen final (PNG o JPEG).
2. Generar metadatos (timestamp, ISO, exposure, frame count, quality metric).
3. Escribir tar.gz en `/downlink/queue/`.

Consumo ~ 2 W. Tiempo: ~ 10 s.

DOWNLINK

Negociacion con OBC para transmitir. El OBC pregunta al payload “tenes datos?” periodicamente; cuando el payload reporta `queue_size > 0`, el OBC programa el proximo COMMS_PASS.

SAFE_MODE

Comando `CMD_SAFE` desde OBC. Payload baja al consumo minimo: ~ 1 W. OV5647 OFF, OLED OFF, solo daemon corriendo para escuchar `CMD_RESUME`.

OTA

Comando `CMD_OTA_COMMIT`. El payload aplica un update de software preinstalado (en un slot B, A/B booting). Si el boot del nuevo slot falla, vuelve automaticamente al slot anterior (rollback).

ERROR

Cualquier fault. Mantiene el modo hasta que llegue `CMD_RESUME` desde tierra. Importante: ERROR NO es lo mismo que `SAFE_MODE` — ERROR mantiene logs detallados del fault para diagnostico; `SAFE_MODE` es solo bajar consumo.

El caso INTISAT

La diferencia entre ERROR y `SAFE_MODE` es clave: ERROR implica “algo se rompio, no toques”, `SAFE_MODE` implica “estoy en pausa voluntaria”. Operacionalmente: si ves ERROR en telemetria, tenes que diagnosticar (descargar logs, hacer un dump). Si ves `SAFE_MODE`, solo mandas `CMD_RESUME` cuando quieras.

9.3. El camino feliz vs los caminos de error

En el diagrama de la figura, las flechas *verdes* forman el “happy path” que va de IDLE a IDLE via todos los pasos del scan. Las flechas *rojas* son los desvios a ERROR. La unica salida de ERROR es `CMD_RESUME` desde tierra.

Capítulo 10

Comparacion contra el INTISAT actual

10.1. Estado del arte por subsistema

Subsistema	Madurez	Estado actual
PAYLOAD	FSM completa (80 %)	Codigo en <code>cubesat/commands.py</code> , daemon corriendo, watchdog systemd, OTA rollback. Falta validacion vibracion + thermal.
OBC	Propuesta (20 %)	Solo este documento. Hardware aun no definido (LPC vs STM32). Sin codigo.
EPS	Propuesta (20 %)	Especificacion teorica. Se evalua GomSpace P31u vs custom. Sin codigo.
ADCS	Propuesta (20 %)	Especificacion teorica. Sensores comprados (magnetometro, giroscopo); falta integrar.
COMMS	Propuesta (20 %)	Especificacion teorica. Radio UHF comprado, antena en diseno mecanico.
MISION	Propuesta (20 %)	Documento describe los 8 modos, falta validar con stakeholders y simular.

10.2. Plan sugerido

Sprint 1 (proximas 2 semanas)

1. **Reunion de matriz** (4 hs). Todos los lideres tecnicos + lider de proyecto. Validar las 45 celdas de la matriz capitulo 4.
2. Firmar la matriz como version 1.0.
3. Cada lider crea un Issue en GitHub para implementar la FSM de su subsistema.

Sprint 2 (siguiente mes)

1. Implementar FSM del OBC en codigo (target: RTOS sobre STM32 o sobre Linux embebido).
2. Simular interaccion OBC + EPS + Payload usando mocks.
3. Definir el protocolo I2C OBC-Payload (mensajes y formato).

Hitos mas alla (3+ meses)

- Bring-up del prototipo OBC.
- Tests de integracion EPS + OBC + Payload.
- Validacion termica (camara de vacio termal).
- Validacion vibracion (mesa shaker).
- Hand-over a operaciones.

Apéndice A

Tabla formal de transiciones (extracto)

Esta es la tabla de transiciones para la FSM de MISION, en el formato estandar (**from**, **trigger**, **condicion**, **to**, **accion**). Es la unica representacion “maquina-leible” de la FSM — todo lo demas es para humanos.

from	trigger	condition	to	action
LAUNCH	deployer_eject	deploy_switch == 1	DETUMBLE	start_bdot
DETUMBLE	rates_low	$ \omega < 1^\circ/\text{s}$	COMMIS.	start_sun_acq
COMMIS.	subsys_OK	all_subsys.NOMINAL	NOMINAL	set_default
NOMINAL	ground_LOS	ground_visible == 1	COMMS_P.	start_target_pt
NOMINAL	sun_low	sun_flux < 5 %	ECLIPSE	inertial_hold
NOMINAL	CMD_START_PL	cmd_rcv	PAYLOAD_OK	start_capture
NOMINAL	anomaly	fdir.trip == 1	SAFE	enter_safe
PAYLOAD_OK	scan_done	scan.status == OK	NOMINAL	exit_capture
COMMS_PASS	pass_done	ground_visible == 0	NOMINAL	stop_target_pt
ECLIPSE	sun_high	sun_flux > 10 %	NOMINAL	resume_normal
SAFE	CMD_RESUME	cmd_rcv & ops_OK	NOMINAL	exit_safe
NOMINAL	alt_low	altitude < 200 km	EOL	start_eol
EOL	reentry	altitude < 80 km	TERM	—

Como se usa esta tabla: cargada en una estructura de datos del OBC (lista de tuplas o diccionario). El loop principal:

```

1 def step(self, trigger, observations):
2     """Ejecuta una transicion si hay match."""
3     for (frm, trg, cond, to, act) in self.transitions:
4         if frm == self.state and trg == trigger:
5             if eval_condition(cond, observations):
6                 self._do_action(act)
7                 self.state = to
8                 return True
9     return False

```

Apéndice B

Referencias para validar

CubeSats con FSM publica

- **OreSat** (Portland State University) — 100 % open source, FSM completa en Python sobre CANopen.
`github.com/oresat`
- **UPSat** (Libre Space Foundation) — primer CubeSat 100 % open en orbita (2017). Documentacion academica.
`gitlab.com/librespacefoundation/upsat`
- **AAUSAT-3 / 4** (Aalborg University) — la mas detallada Operations Concept document del mundo abierto.
`space.aau.dk`

Frameworks de FSM espacial

- **NASA cFS** (Core Flight System) — framework completo de flight software con apps modulares.
`github.com/nasa/cFS`
- **NASA cFS Limit Checker** — modulo de FDIR plantilla.
`github.com/nasa/lc`
- **NASA F'** (FPrime) — genera FSM ejecutable desde XML. Usado en MarCO, Ingenuity (Marte).
`nasa.github.io/fprime`

Estandares

- **ECSS-E-ST-70-11C** — ESA Space Segment Operability. Define el marco formal de modos y transiciones para satelites europeos.
`ecss.nl/ecss-e-st-70-11c`
- **ECSS-E-ST-70-31C** — Ground Systems and Operations. Procedimientos asociados a cada modo.
- **CCSDS Mission Operations** — Consultative Committee for Space Data Systems. Protocolos de TM/TC para coordinar estado entre tierra y satelite.
`ccsds.org`

Lectura academica recomendada

- Wertz, J. R. et al. *Space Mission Engineering: The New SMAD*. Microcosm Press, 2011. Capitulo 14 (operations concept).
- Larson, W., Wertz, J. *Space Mission Analysis and Design*. Capitulo 11 (mission operations).
- Eickhoff, J. *Onboard Computers, Onboard Software and Satellite Operations: An Introduction*. Springer, 2012.